

Java and Spring Boot Development Guide

Table of Contents

1. [Core Java Fundamentals](#core-java)
2. [Spring Framework Basics](#spring-basics)
3. [Microservices Architecture](#microservices)
4. [Database Optimization](#database)
5. [Testing & Quality Assurance](#testing)

Core Java

Object-Oriented Programming (OOP)

Classes and Objects

A class is a blueprint for creating objects. Objects are instances of classes that contain state and behavior.

```
```java
```

```
Public class Car {

 // State (attributes)

 Private String model;

 Private int year;

 // Behavior (methods)

 Public void startEngine() {
 System.out.println("Engine started");
 }
}
```

```
```
```

Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

Key Points:

- Use 'extends' keyword for inheritance
- Java supports single inheritance
- All classes inherit from Object class
- Protected members are accessible in child classes

Example:

```
```java
Public class ElectricCar extends Car {
 Private int batteryLevel;

 Public void charge() {
 batteryLevel = 100;
 }
}
```
```

Interfaces

Interfaces define a contract of methods that implementing classes must provide.

```
```java
Public interface Vehicle {
 Void accelerate();
}
```

```
 void brake();
}
```

```
public class Car implements Vehicle {
 @Override
 public void accelerate() {
 // Implementation
 }
}
```

```
 @Override
 public void brake() {
 // Implementation
 }
}
...
}
```

### ### Collections Framework

#### #### List Interface

Lists maintain ordered collections of elements.

ArrayList:

- Backed by dynamic array
- Fast random access
- $O(1)$  for add/get
- $O(n)$  for insert/remove

```
```java
```

```
List<String> list = new ArrayList<>();  
List.add("First");  
List.add("Second");  
` ` `
```

LinkedList:

- Doubly-linked list implementation
- $O(1)$ for add/remove at ends
- $O(n)$ for random access
- Better for frequent insertions/deletions

Map Interface

Maps store key-value pairs.

HashMap:

- $O(1)$ average case for put/get
- Allows null keys and values
- No ordering guaranteed

```
` ` ` java
```

```
Map<String, Integer> map = new HashMap<>();  
Map.put("One", 1);  
Map.put("Two", 2);  
` ` `
```

TreeMap:

- Red-black tree implementation
- $O(\log n)$ for operations

- Keys stored in sorted order
- No null keys allowed

Java 8+ Features

Lambda Expressions

Anonymous functions that can be passed as arguments.

```
```java
```

```
// Traditional anonymous class
```

```
Runnable r1 = new Runnable() {
```

```
 @Override
```

```
 Public void run() {
```

```
 System.out.println("Running");
```

```
 }
```

```
};
```

```
// Lambda expression
```

```
Runnable r2 = () -> System.out.println("Running");
```

```
```
```

Stream API

Provides functional-style operations on streams of elements.

```
```java
```

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
```

```
// Filter and map operation
```

```
List<String> filteredNames = names.stream()
 .filter(name -> name.length() > 3)
 .map(String::toUpperCase)
 .collect(Collectors.toList());
` ``
```

## ## Spring Framework

### ### Core Concepts

#### #### Dependency Injection

Spring manages object dependencies through injection.

```
` `` java
@Service
Public class UserService {
 Private final UserRepository userRepository;

 @Autowired
 Public UserService(UserRepository userRepository) {
 This.userRepository = userRepository;
 }
}
` ``
```

#### #### Bean Lifecycle

1. Instantiation
2. Property Population

3. BeanNameAware
4. BeanFactoryAware
5. ApplicationContextAware
6. PreInitialization
7. InitializingBean
8. Custom Init Method
9. Post Initialization
10. DisposableBean
11. Custom Destroy Method

```
```java
```

```
@Component
```

```
Public class MyBean implements InitializingBean, DisposableBean {
```

```
    @PostConstruct
```

```
    Public void init() {
```

```
        // Initialization code
```

```
    }
```

```
    @PreDestroy
```

```
    Public void cleanup() {
```

```
        // Cleanup code
```

```
    }
```

```
}
```

```
```
```

### Spring MVC

#### Controller Pattern

```

```java

@RestController
@RequestMapping("/api/users")

Public class UserController {

    Private final UserService userService;


    @GetMapping("/{id}")
    Public ResponseEntity<User> getUser(@PathVariable Long id) {

        Return ResponseEntity.ok(userService.findById(id));

    }


    @PostMapping
    Public ResponseEntity<User> createUser(@RequestBody User user) {

        Return ResponseEntity.status(201)

            .body(userService.create(user));

    }

}
```

```

#### #### Exception Handling

```

```java

@ControllerAdvice

Public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)

    Public ResponseEntity<ErrorResponse>
    handleUserNotFound(UserNotFoundException ex) {

        Return ResponseEntity.status(404)

            .body(new ErrorResponse(ex.getMessage()));

    }

}
```

```

```
 }
}
...
```

## Microservices

### Service Communication

#### REST Template

```
```java  
  
@Service  
  
Public class UserClient {  
  
    Private final RestTemplate restTemplate;  
  
  
    Public User getUser(Long id) {  
  
        Return restTemplate.getForObject(  
            http://user-service/users/ + id,  
            User.class  
        );  
    }  
}  
...
```

Circuit Breaker

```
```java  

@CircuitBreaker(name = "userService")

@Retry(name = "userService")

Public User getUserWithResilience(Long id) {
```

```
 Return userClient.getUser(id);
}
...
```

### ### Monitoring

#### #### Logging Pattern

```
```java  
@Slf4j  
@Service  
Public class OrderService {  
    Public Order processOrder(Order order) {  
        Log.info("Processing order: {}", order.getId());  
        Try {  
            // Process order  
            Log.debug("Order processed successfully");  
            Return order;  
        } catch (Exception e) {  
            Log.error("Error processing order: {}", e.getMessage(), e);  
            Throw e;  
        }  
    }  
}  
}  
...
```

Database Optimization

JPA Best Practices

Entity Design

```
```java
@Entity
@Table(name = "users")
Public class User {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 Private Long id;

 @Column(nullable = false)
 Private String name;

 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
 Private Set<Order> orders = new HashSet<>();
}
```
```

Query Optimization

```
```java
@Repository
Public interface UserRepository extends JpaRepository<User, Long> {

 @Query("SELECT u FROM User u LEFT JOIN FETCH u.orders WHERE u.id = :id")
 Optional<User> findByIdWithOrders(@Param("id") Long id);
}
```
```

Testing

Unit Testing

Service Layer Test

```
```java
```

```
@ExtendWith(MockitoExtension.class)
```

```
Class UserServiceTest {
```

```
 @Mock
```

```
 Private UserRepository userRepository;
```

```
 @InjectMocks
```

```
 Private UserService userService;
```

```
 @Test
```

```
 Void shouldCreateUser() {
```

```
 // Given
```

```
 User user = new User("Test User");
```

```
 When(userRepository.save(any(User.class))).thenReturn(user);
```

```
 // When
```

```
 User created = userService.createUser(user);
```

```
 // Then
```

```
 assertNotNull(created);
```

```
 assertEquals("Test User", created.getName());
```

```
 verify(userRepository).save(any(User.class));
```

```
 }
```

```
}
```

```

Integration Testing

API Test

```java

@SpringBootTest(webEnvironment = WebEnvironment.RANDOM\_PORT)

Class UserControllerIT {

    @Autowired

    Private TestRestTemplate restTemplate;

    @Test

    Void shouldCreateUser() {

        // Given

        UserRequest request = new UserRequest("Test User");

        // When

        ResponseEntity<UserResponse> response = restTemplate.postForEntity(

            "/api/users",

            Request,

            UserResponse.class

        );

        // Then

        assertEquals(201, response.getStatusCodeValue());

        assertNotNull(response.getBody());

        assertEquals("Test User", response.getBody().getName());

    }

```
}
` ``
```

## ## Best Practices and Guidelines

### ### Code Quality

#### 1. Follow SOLID Principles

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

#### 2. Clean Code Guidelines

- Meaningful names
- Small methods and classes
- DRY (Don't Repeat Yourself)
- Comments only when necessary
- Proper exception handling

#### 3. Performance Optimization

- Use appropriate data structures
- Optimize database queries
- Implement caching where beneficial
- Monitor and profile application

### ### Security

## 1. Input Validation

- Validate all user input
- Use prepared statements
- Implement proper authentication
- Apply authorization checks

## 2. API Security

- Use HTTPS
- Implement rate limiting
- Apply proper CORS settings
- Use security headers

## ## Conclusion

This guide covers the fundamental and advanced concepts of Java and Spring Boot development. Regular practice and hands-on implementation of these concepts will help build expertise. Remember to:

1. Start with basics and gradually move to advanced topics
2. Practice regularly with hands-on coding
3. Review and refactor code frequently
4. Stay updated with latest features and best practices
5. Participate in code reviews and discussions

Keep this guide as a reference while working on projects and continuously update your knowledge as new features and best practices emerge.